Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen[1]

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway[1]

April 30

Plans for the week of April 28-May 2

1. Classical Support Vector Machines, reminder from last week
2. Classical Kernels and transition to Quantum Kernels
3. Quantum Support Vector Machines

Reading recommendations

1. For classical Support Vector Machines, see lecture notes by Morten Hjorth-Jensen at https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/chapter5.html
2. Claudio Conti, Quantum Machine Learning (Springer), sections 1.5-1.12 and chapter 2, see https://link.springer.com/book/10.1007/978-3-031-44226-1.
3. See also Maria Schuld's article at https://arxiv.org/abs/2101.11020.
4. For coding, we included examples using PennyLane at https://pennylane.ai/ and Qiskit, see https://qiskit-community.github.io/qiskit-machine-learning/

What is Quantum Machine Learning?

Quantum Machine Learning (QML) integrates quantum computing with machine learning algorithms to exploit quantum advantages. It explores how quantum computing can enhance classical machine learning.

Motivation:

1. High-dimensional Hilbert spaces for better feature representation.
2. Quantum parallelism for faster computation.
3. Quantum entanglement for richer data encoding.

Quantum Speedups in ML

Why Quantum?

1. **Quantum Parallelism:** Process multiple states simultaneously.
2. **Quantum Entanglement:** Correlated states for richer information.
3. **Quantum Interference:** Constructive and destructive interference to enhance solutions.

Challenges in Quantum Machine Learning

Quantum Hardware Limitations:

1. Noisy Intermediate-Scale Quantum (NISQ) devices.
2. Decoherence and limited qubit coherence times.

Data Encoding:

1. Efficient embedding of classical data into quantum states.

Scalability:

1. Difficult to scale circuits to large datasets.

Support vector machines

As discussed last week, a central model in classical supervised learning is the support vector machine (SVM), which is a max-margin classifier. SVMs are widely used for binary classification and have extensions to regression problems as well. They build on statistical learning theory and are known for finding decision boundaries with maximal margin. In particular, SVMs can perform non-linear classification by employing the kernel trick, which implicitly maps data into a high-dimensional feature space via a kernel function.

The quantum SVM

A Quantum Support Vector Machine (QSVM) replaces the classical kernel or feature map with a quantum procedure. In QSVM, classical data points $\boldsymbol{x}$ are encoded into quantum states $|\phi(\boldsymbol{x})\rangle$ via a quantum feature map (a parameterized quantum circuit). The inner product (overlap) between two such states serves as a quantum kernel, measuring data similarity in a high-dimensional Hilbert space. Many quantum models, including **quantum neural networks**, ultimately behave like kernel methods: they analyze data in exponentially-large Hilbert spaces, accessible only through state overlaps, see for example Maria Schuld, Supervised quantum machine learning models are kernel methods, at arXiv:2101.11020

More on quantun SVMs

Schuld shows that supervised quantum models can be reformulated as SVMs whose kernels compute the distance (inner product) between data-encoded quantum states . Thus, QSVMs can leverage the expressive power of quantum state space to potentially separate data that classical kernels struggle with.

Kernels and more

In classical SVMs, kernels help with non-linear data separations. Quantum computers can speed up the computation of complex kernel evaluations by efficiently simulating an inner product in an exponentially large Hilbert space.

Quantum Support Vector Machines (QSVMs)

Quantum Kernel Estimation:

1. Maps classical data to a quantum Hilbert space.
2. Quantum kernel measures similarity in high-dimensional space.

Quantum-enhanced kernel:

$$K(\boldsymbol{x}, \boldsymbol{x}') = |\langle \psi(\boldsymbol{x}) | \psi(\boldsymbol{x}') \rangle|^2$$

Here, $|\phi(\boldsymbol{x})\rangle$ is the quantum state encoding of the classical data $\boldsymbol{x}$.
Advantage: Potentially exponential speedup over classical SVMs.

Quantum Neural Networks (QNNs)

Quantum Neural Networks replace classical neurons with parameterized quantum circuits.

Key Concepts:

1. Quantum Gates as Activation Functions.
2. Variational Quantum Circuits (VQCs) for optimization.

Parameterized Quantum Circuit:

$$U(\theta) = \prod_i R_y(\theta_i) \cdot CNOT \cdot R_x(\theta_i)$$

Advantage:Quantum gradients enable exploration of non-convex landscapes.

Quantum Boltzmann Machines (QBMs)

Quantum Boltzmann Machines leverage quantum mechanics to sample from a probability distribution.

1. Quantum tunneling aids in escaping local minima.
2. Quantum annealing for optimization problems.

Quantum Hamiltonian:

$$H = -\sum_i b_i \sigma_i^z - \sum_{ij} w_{ij} \sigma_i^z \sigma_j^z$$

Advantage: Efficient sampling in complex probability distributions.

Key properties of SVMs

Support vector machines use only the support vectors (training points on the margin) to define the classifier, making the decision function sparse and often memory-efficient. They are robust to overfitting when properly regularized (max-margin helps generalization) .
However, training SVMs can be costly for very large datasets, as one must solve a quadratic program that scales roughly as $O(N^3)$ in naive implementations. Evaluating the kernel (especially for complex kernels or large feature dimension) can also be expensive. In fact, "kernel methods for machine learning are ubiquitous for pattern recognition, with SVMs being the most well-known method..." , but they face limitations when feature spaces become large and kernel computations costly. This sets the stage for using quantum computers to estimate kernels or implement SVMs more efficiently.

What is a hyperplane?

The aim of the SVM algorithm is to find a hyperplane in a p-dimensional space, where p is the number of features that distinctly classifies the data points.
In a p-dimensional space, a hyperplane is what we call an affine subspace of dimension of $p - 1$. As an example, in two dimension, a hyperplane is simply as straight line while in three dimensions it is a two-dimensional subspace, or stated simply, a plane.

Introducing the margin

Last week we defined the so-called margin between classes of a binary problem.
We derived aa margin M with $\boldsymbol{w}$ normalized to $||\boldsymbol{w}|| = 1$ subject to the condition

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) \geq M \; \forall i = 1, 2, \ldots, p.$$

All points are thus at a signed distance from the decision boundary defined by the line L. The parameters b and w_1 and w_2 define this line.

Largest value M

We seek the largest value M defined by

$$\frac{1}{||\boldsymbol{w}||} y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) \geq M \, \forall i = 1, 2, \ldots, n,$$

or just

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) \geq M||\boldsymbol{w}|| \, \forall i.$$

If we scale the equation so that $||\boldsymbol{w}|| = 1/M$, we have to find the minimum of $\boldsymbol{w}^T \boldsymbol{w} = ||\boldsymbol{w}||$ (the norm) subject to the condition

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) \geq 1 \, \forall i.$$

We have thus defined our margin as the invers of the norm of $\boldsymbol{w}$. We want to minimize the norm in order to have a as large as possible margin M.

Setting up the problem

In order to solve the above problem, we define the following Lagrangian function to be minimized

$$\mathcal{L}(\lambda, b, \boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_{i=1}^{n} \lambda_i \left[y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1 \right],$$

where λ_i is a so-called Lagrange multiplier subject to the condition $\lambda_i \geq 0$.

Setting up derivaties

Taking the derivatives with respect to b and $\boldsymbol{w}$ we obtain

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_i \lambda_i y_i = 0,$$

and

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}} = 0 = \boldsymbol{w} - \sum_i \lambda_i y_i \boldsymbol{x}_i.$$

Inserting these constraints into the equation for $\mathcal{L}$ we obtain

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2}\sum_{ij}^{n} \lambda_i \lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j,$$

subject to the constraints $\lambda_i \geq 0$ and $\sum_i \lambda_i y_i = 0$.

Karush-Kuhn-Tucker condition

We must in addition satisfy the Karush-Kuhn-Tucker (KKT) condition

$$\lambda_i \left[y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) - 1 \right] \ \forall i.$$

1. If $\lambda_i > 0$, then $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1$ and we say that x_i is on the boundary.
2. If $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) > 1$, we say x_i is not on the boundary and we set $\lambda_i = 0$.

When $\lambda_i > 0$, the vectors $\boldsymbol{x}_i$ are called support vectors. They are the vectors closest to the line (or hyperplane) and define the margin M.

The problem to solve

We can rewrite

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2}\sum_{ij}^{n} \lambda_i\lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j,$$

and its constraints in terms of a matrix-vector problem where we minimize w.r.t. λ the following problem

$$\frac{1}{2}\boldsymbol{\lambda}^T \begin{bmatrix} y_1y_1\boldsymbol{x}_1^T\boldsymbol{x}_1 & y_1y_2\boldsymbol{x}_1^T\boldsymbol{x}_2 & \dots & \dots & y_1y_n\boldsymbol{x}_1^T\boldsymbol{x}_n \\ y_2y_1\boldsymbol{x}_2^T\boldsymbol{x}_1 & y_2y_2\boldsymbol{x}_2^T\boldsymbol{x}_2 & \dots & \dots & y_1y_n\boldsymbol{x}_2^T\boldsymbol{x}_n \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ y_ny_1\boldsymbol{x}_n^T\boldsymbol{x}_1 & y_ny_2\boldsymbol{x}_n^T\boldsymbol{x}_2 & \dots & \dots & y_ny_n\boldsymbol{x}_n^T\boldsymbol{x}_n \end{bmatrix} \boldsymbol{\lambda} - \mathbb{1}\boldsymbol{\lambda},$$

subject to $\boldsymbol{y}^T\boldsymbol{\lambda} = 0$. Here we defined the vectors $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_n]$ and $\boldsymbol{y} = [y_1, y_2, \dots, y_n]$.

The last steps

Solving the above problem, yields the values of λ_i. To find the coefficients of your hyperplane we need simply to compute

$$\boldsymbol{w} = \sum_i \lambda_i y_i \boldsymbol{x}_i.$$

With our vector $\boldsymbol{w}$ we can in turn find the value of the intercept b (here in two dimensions) via

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1,$$

resulting in

$$b = \frac{1}{y_i} - \boldsymbol{w}^T \boldsymbol{x}_i,$$

or if we write it out in terms of the support vectors only, with N_s being their number, we have

$$b = \frac{1}{N_s} \sum_{j \in N_s} \left(y_j - \sum_{i=1}^{n} \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x}_j \right).$$

Classifier equations

With our hyperplane coefficients we can use our classifier to assign any observation by simply using

$$y_i = \text{sign}(\boldsymbol{w}^T \boldsymbol{x}_i + b).$$

Below we discuss how to find the optimal values of λ_i. Before we proceed however, we discuss now the so-called soft classifier.

A soft classifier

Till now, the margin is strictly defined by the support vectors. This defines what is called a hard classifier, that is the margins are well defined.
Suppose now that classes overlap in feature space, as shown in the figure here. One way to deal with this problem before we define the so-called **kernel approach**, is to allow a kind of slack in the sense that we allow some points to be on the wrong side of the margin. We introduce thus the so-called **slack** variables $\boldsymbol{\xi} = [\xi_1, x_2, \ldots, x_n]$ and modify our previous equation

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) = 1,$$

to

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) = 1 - \xi_i,$$

with the requirement $\xi_i \geq 0$. The total violation is now $\sum_i \xi$.

Misclassification

The value ξ_i in the constraint the last constraint corresponds to the amount by which the prediction $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1$ is on the wrong side of its margin. Hence by bounding the sum $\sum_i \xi_i$, we bound the total amount by which predictions fall on the wrong side of their margins.
Misclassifications occur when $\xi_i > 1$. Thus bounding the total sum by some value C bounds in turn the total number of misclassifications.

Soft optmization problem

This has in turn the consequences that we change our optmization problem to finding the minimum of

$$\mathcal{L} = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_{i=1}^{n}\lambda_i\left[y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - (1-\xi)\right] + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\gamma_i\xi_i,$$

subject to

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) = 1 - \xi_i \; \forall i,$$

with the requirement $\xi_i \geq 0$.

Derivatives with respect to b and $\boldsymbol{w}$

Taking the derivatives with respect to b and $\boldsymbol{w}$ we obtain

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_i \lambda_i y_i = 0,$$

and

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}} = 0 = \boldsymbol{w} - \sum_i \lambda_i y_i \boldsymbol{x}_i,$$

and

$$\lambda_i = C - \gamma_i \; \forall i.$$

New constraints

Inserting these constraints into the equation for $\mathcal{L}$ we obtain the same equation as before

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2}\sum_{ij}^{n} \lambda_i\lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j,$$

but now subject to the constraints $\lambda_i \geq 0$, $\sum_i \lambda_i y_i = 0$ and $0 \leq \lambda_i \leq C$. We must in addition satisfy the Karush-Kuhn-Tucker condition which now reads

$$\lambda_i \left[y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) - (1 - \xi_) \right] = 0 \; \forall i,$$

$$\gamma_i \xi_i = 0,$$

and

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) - (1 - \xi_) \geq 0 \; \forall i.$$

Kernels and non-linearity

The cases we have studied till now, were all characterized by two classes with a close to linear separability. The classifiers we have described so far find linear boundaries in our input feature space. It is possible to make our procedure more flexible by exploring the feature space using other basis expansions such as higher-order polynomials, wavelets, splines etc.

If our feature space is not easy to separate, as shown in the figure here, we can achieve a better separation by introducing more complex basis functions. The ideal would be, as shown in the next figure, to, via a specific transformation to obtain a separation between the classes which is almost linear.

The change of basis, from $x \rightarrow z = \phi(x)$ leads to the same type of equations to be solved, except that we need to introduce for example a polynomial transformation to a two-dimensional training set.

```
import numpy as np
import os

np.random.seed(42)

# To plot pretty figures
```

Kernel trick

We note that this is nothing but the dot product of the two original vectors $(\boldsymbol{x}_i^T \boldsymbol{x}_j)^2$. Instead of thus computing the product in the Lagrangian of $\boldsymbol{z}_i^T \boldsymbol{z}_j$ we simply compute the dot product $(\boldsymbol{x}_i^T \boldsymbol{x}_j)^2$. This leads to the so-called kernel trick and the result leads to the same as if we went through the trouble of performing the transformation $\phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j)$ during the SVM calculations.

The problem to solve

Using our definition of the kernel We can rewrite again the Lagrangian

$$\mathcal{L} = \sum_i \lambda_i - \frac{1}{2}\sum_{ij}^{n} \lambda_i \lambda_j y_i y_j \boldsymbol{z}_i^T \boldsymbol{z}_j,$$

subject to the constraints $\lambda_i \geq 0$, $\sum_i \lambda_i y_i = 0$ in terms of a convex optimization problem

$$\frac{1}{2}\boldsymbol{\lambda}^T \begin{bmatrix} y_1 y_1 K(\boldsymbol{x}_1, \boldsymbol{x}_1) & y_1 y_2 K(\boldsymbol{x}_1, \boldsymbol{x}_2) & \dots & \dots & y_1 y_n K(\boldsymbol{x}_1, \boldsymbol{x}_n) \\ y_2 y_1 K(\boldsymbol{x}_2, \boldsymbol{x}_1) & y_2 y_2 (\boldsymbol{x}_2, \boldsymbol{x}_2) & \dots & \dots & y_1 y_n K(\boldsymbol{x}_2, \boldsymbol{x}_n) \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ y_n y_1 K(\boldsymbol{x}_n, \boldsymbol{x}_1) & y_n y_2 K(\boldsymbol{x}_n \boldsymbol{x}_2) & \dots & \dots & y_n y_n K(\boldsymbol{x}_n, \boldsymbol{x}_n) \end{bmatrix} \boldsymbol{\lambda} - \mathbb{1}\boldsymbol{\lambda},$$

subject to $\boldsymbol{y}^T \boldsymbol{\lambda} = 0$. Here we defined the vectors $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_n]$ and $\boldsymbol{y} = [y_1, y_2, \dots, y_n]$. If we add the slack constants this leads to the additional constraint $0 \leq \lambda_i \leq C$.

Convex optimization

We can rewrite this (see the solutions below) in terms of a convex optimization problem of the type

$$\begin{aligned} &\min_\lambda \ \frac{1}{2}\boldsymbol{\lambda}^T\boldsymbol{P}\boldsymbol{\lambda} + \boldsymbol{q}^T\boldsymbol{\lambda}, \\ &\text{subject to } \ \boldsymbol{G}\boldsymbol{\lambda} \preceq \boldsymbol{h} \ \ \wedge \boldsymbol{A}\boldsymbol{\lambda} = f. \end{aligned}$$

Below we discuss how to solve these equations. Here we note that the matrix $\boldsymbol{P}$ has matrix elements $p_{ij} = y_i y_j K(\boldsymbol{x}_i, \boldsymbol{x}_j)$. Given a kernel K and the targets y_i this matrix is easy to set up. The constraint $\boldsymbol{y}^T\boldsymbol{\lambda} = 0$ leads to $f = 0$ and $\boldsymbol{A} = \boldsymbol{y}$. How to set up the matrix $\boldsymbol{G}$ is discussed later. Here note that the inequalities $0 \leq \lambda_i \leq C$ can be split up into $0 \leq \lambda_i$ and $\lambda_i \leq C$. These two inequalities define then the matrix $\boldsymbol{G}$ and the vector $\boldsymbol{h}$.

Different kernels

There are several popular kernels being used. These are

1. Linear: $K(\boldsymbol{v}, \boldsymbol{w}) = \boldsymbol{v}^T \boldsymbol{w}$,
2. Polynomial: $K(\boldsymbol{v}, \boldsymbol{w}) = (\boldsymbol{v}^T \boldsymbol{w} + \gamma)^d$,
3. Gaussian Radial Basis Function:
   $K(\boldsymbol{v}, \boldsymbol{w}) = \exp\left(-\gamma ||\boldsymbol{v} - \boldsymbol{w}||^2\right)$,
4. Tanh: $K(\boldsymbol{v}, \boldsymbol{w}) = \tanh\left(\boldsymbol{v}^T \boldsymbol{w} + \gamma\right)$,

and many other ones.
The kernel trick involves implicitly mapping the input data into a higher-dimensional feature space where a linear separation is possible. Instead of explicitly transforming each data point using a mapping function, the kernel function computes the inner product directly. This approach avoids the computational cost of handling high-dimensional spaces explicitly.

Mercer's theorem

An important theorem for us is Mercer's theorem. The theorem states that if a kernel function K is symmetric, continuous and leads to a positive semi-definite matrix $\boldsymbol{P}$ then there exists a function ϕ that maps $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ into another space (possibly with much higher dimensions) such that

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j).$$

So you can use K as a kernel since you know ϕ exists, even if you don't know what ϕ is.
Note that some frequently used kernels (such as the Sigmoid kernel) don't respect all of Mercer's conditions, yet they generally work well in practice.

The moons example

```
from __future__ import division, print_function, unicode_literals

import numpy as np
np.random.seed(42)

import matplotlib
import matplotlib.pyplot as plt
plt.rcParams['axes.labelsize'] = 14
plt.rcParams['xtick.labelsize'] = 12
plt.rcParams['ytick.labelsize'] = 12


from sklearn.svm import SVC
from sklearn import datasets



from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC


from sklearn.datasets import make_moons
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

def plot_dataset(X, y, axes):
    plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
    plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^")
```

The Iris data

```
# import the required packages
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score,classification_report,f1_sc
iris = load_iris()
dataset=pd.DataFrame(iris.data,columns=iris.feature_names)
dataset["target"]=iris.target
dataset
features= dataset[dataset.columns[1:4]]
target=dataset["target"]
# split the data into training set and testing set
X_train,X_test,Y_train,Y_test=train_test_split(features,target,test_si
# Scale the data
from sklearn.preprocessing import MinMaxScaler

minmax=MinMaxScaler()
X_train=minmax.fit_transform(X_train)
X_test=minmax.transform(X_test)
X_test=np.clip(X_test,0,1)
import time
start_time=time.time()
svm_model=SVC(kernel="rbf")
svm_model.fit(X_train, Y_train)
end_time=time.time()
print(end_time-start_time)
```

Quantum SVMs

The idea of a classical SVM is that we have a set of points that are in either one group or another and we want to find a line that separates these two groups. This line can be linear, but it can also be much more complex, which can be achieved by the use of Kernels.

Basic steps: Data Encoding (Quantum Feature Mapping)

Mapping to Quantum States: Input data is first encoded into quantum states. This process is akin to a quantum feature map, where classical information is represented in the quantum Hilbert space.

Basic steps: Quantum Kernel

In many QSVM implementations, this mapping enables the computation of a kernel matrix, where the similarity (or inner product) between quantum states is evaluated. This quantum kernel can capture complex relationships in data that might be challenging for classical kernels.

Basic steps: Quantum Circuit Processing

Quantum Gates and Circuits: Once data is encoded, a series of quantum gates (forming a quantum circuit) is applied. These gates manipulate the quantum states to perform the equivalent of the SVM algorithm's optimization and decision-making steps.

Basic steps: Measurement

Outcome: After the quantum operations, a measurement is performed on the output state. The result of this measurement provides the classification outcome, analogous to deciding on which side of the hyperplane a data point lies.

Quantum Kernels and Feature Maps

A core idea of QSVM is to use a quantum feature map that encodes classical input $\boldsymbol{x}$ into a quantum state $|\phi(\boldsymbol{x})\rangle$ in an exponentially large Hilbert space.
Mathematically, a quantum feature map is a unitary circuit $U(\boldsymbol{x})$ acting on an n-qubit register initialized to $|0\rangle^{\otimes n}$, producing

$$|\phi(\boldsymbol{x})\rangle = U(\boldsymbol{x})|0\rangle^{\otimes n}.$$

Input dependence

This state depends on the input $\boldsymbol{x} \in \mathbb{R}^d$. The choice of $U(\boldsymbol{x})$ is analogous to choosing a classical feature mapping $\phi(\boldsymbol{x})$ in kernel methods. Common quantum feature maps include amplitude encoding (embedding data amplitudes in the state vector) and angle encoding (rotations whose angles are functions of $\boldsymbol{x}$) .
Once data are encoded as quantum states, one defines a quantum kernel as an overlap between states. A simple definition is the fidelity:

$$K(\boldsymbol{x}, \boldsymbol{x\prime}) = \left|\langle \phi(\boldsymbol{x}) \mid \phi(\boldsymbol{x\prime})\rangle\right|^2.$$

Quantum kernels

That is, the kernel is the squared magnitude of the inner product of the two quantum states. Another common (unnormalized) version is

$$K'(\boldsymbol{x}, \boldsymbol{x}') = \langle \phi(\boldsymbol{x}) | \phi(\boldsymbol{x}') \rangle,$$

but measuring this amplitude directly can be difficult due to the absolute value. Many QSVM implementations (and theoretical definitions) use the squared overlap. In any case, the kernel measures similarity: if $|\phi(\boldsymbol{x})\rangle$ and $|\phi(\boldsymbol{x}')\rangle$ are close in Hilbert space, $k(\boldsymbol{x}, \boldsymbol{x}')$ is large.

What is a quantum kernel?

A quantum kernel is a function that measures the similarity between two quantum states, which are obtained by applying a quantum feature map to classical data. In other words, each classical vector $\boldsymbol{x}$ is mapped to a quantum state $|\phi(\boldsymbol{x})\rangle$ via a feature map, and the kernel is defined by the inner product of these states . Concretely, one may write

$$K_{ij} = K(\boldsymbol{x}_i, \boldsymbol{x}_j) \;=\; \left|\langle \phi(\boldsymbol{x}_i) \mid \phi(\boldsymbol{x}_j)\rangle\right|^2.$$

This forms a positive semidefinite kernel matrix K on the dataset, just as in classical SVMs.

What will we need in the case of a quantum computer?

We will have to translate the classical data point $\vec{x}$ into a quantum datapoint $|\Phi(\vec{x})\rangle$. This can be achieved by a circuit $\mathcal{U}_{\Phi(\vec{x})}|0\rangle$. Here $\Phi()$ could be any classical function applied on the classical data $\vec{x}$.

We need a parameterized quantum circuit $W(\theta)$ that processes the data in a way that in the end we can apply a measurement that returns a classical value -1 or 1 for each classical input $\vec{x}$ that indentifies the label of the classical data.

The most general ansatz

Following these steps we can define an ansatz for this kind of problem which is

$$W(\theta)\mathcal{U}_{\Phi}(\vec{x})|0\rangle.$$

These kind of ansatzes are called quantum variational circuits.

Quantum SVM

In the case of a quantum SVM we will only use the quantum feature maps

$$\mathcal{U}_{\Phi(\vec{x})},$$

to translate the classical data into quantum states and build the Kernel of the SVM out of these quantum states. After calculating the Kernel matrix on the quantum computer we can train the Quantum SVM the same way as the classical SVM.

Defining the Quantum Kernel

The idea of the quantum kernel is exactly the same as in the classical case. We take the inner product

$$K(\vec{x}, \vec{z}) = |\langle \Phi(\vec{x}) | \Phi(\vec{z}) \rangle|^2 = \langle 0^n | \mathcal{U}^t_{\Phi(\vec{x})} \mathcal{U}_{\Phi(\vec{z})} | 0^n \rangle,$$

but now with the quantum feature maps

$$\mathcal{U}_{\Phi(\vec{x})}.$$

The idea is that if we choose a quantum feature maps that is not easy to simulate with a classical computer we might obtain a quantum advantage.

Side note

There is no proof yet that the QSVM brings a quantum advantage, but the argument the authors of https://arxiv.org/pdf/1804.11326.pdf make, is that there is for sure no advantage if we use feature maps that are easy to simulate classically, because then we would not need a quantum computer to construct the Kernel.

Feature map

For the feature maps we use the ansatz

$$\mathcal{U}_{\Phi(x)} = U_{\Phi(x)} \otimes H^{\otimes n},$$

where

$$U_{\Phi(x)} = \exp\left(i \sum_{S \in n} \phi_S(x) \prod_{i \in S} Z_i \right),$$

which simplifies a lot when we (like in https://arxiv.org/pdf/1804.11326.pdf) only consider $S \leq 2$ interactions, which means we only let two qubits interact at a time. For $S \leq 2$ the product $\prod_{i \in S}$ only leads to interactions $Z_i Z_j$ and non interacting terms Z_i. And the sum $\sum_{S \in n}$ over all these terms that are possible with n qubits.

Power of quantum kernels

The power of quantum kernels comes from the exponentially large feature space. An n-qubit state has dimension 2^n, so even a simple feature map using n qubits corresponds to an embedding of $\boldsymbol{x}$ into $\mathbb{C}^{2^n}$. Entangling gates in $U(\boldsymbol{x})$ allow highly non-linear features. In principle, a carefully chosen quantum feature map can capture complex structures in data that are hard to model classically. For example, Havlíček et al. used a feature map involving Pauli-Z rotations and controlled-Z gates to embed data into a 2^n-dimensional space . They note that classical kernels become expensive as feature dimensions grow, while quantum circuits naturally operate in these large spaces .

Estimating quantum kernels

We must be able to estimate the quantum kernel in practice. Given two data vectors $\boldsymbol{x}$ and $\boldsymbol{x}'$, we need to compute (or measure) $|\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$. This can be done on a quantum computer by preparing $|\phi(\boldsymbol{x})\rangle$ and $|\phi(\boldsymbol{x}')\rangle$ in some interference experiment. For instance, one can prepare $|\phi(\boldsymbol{x})\rangle$, then apply $U(\boldsymbol{x}')^\dagger$, and measure the probability of returning to the $|0\rangle^{\otimes n}$ state. That probability is exactly $|\langle 0|U(\boldsymbol{x})^\dagger U(\boldsymbol{x}')|0\rangle|^2 = |\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$. In PennyLane, this is conveniently done by defining a QNode circuit that encodes $\boldsymbol{x}$, then encodes $\boldsymbol{x}'$ inversely (via the adjoint).

Code example

For example, using PennyLane's AngleEmbedding template, we can write:

```
import pennylane as qml
from pennylane import numpy as np
dev = qml.device('default.qubit', wires=2)
@qml.qnode(dev)
def kernel_circuit(x1, x2):
    qml.templates.AngleEmbedding(x1, wires=[0,1])
    qml.adjoint(qml.templates.AngleEmbedding)(x2, wires=[0,1])
    return qml.probs(wires=[0,1])[0]
```

What happens

Here, **AngleEmbedding(x1)** encodes the vector **x1** into two qubits via rotations, and qml.adjoint applies the inverse embedding for **x2**. The output probability of measuring the $|00\rangle$ state is exactly the squared overlap of the two embedded states. Using this kernel function, PennyLane can compute the kernel matrix $K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$ efficiently for training. In fact, PennyLane provides a utility qml.kernels.kernel_matrix to evaluate such kernels systematically .

Quantum feature map

A quantum feature map $\boldsymbol{x} \mapsto |\phi(\boldsymbol{x})\rangle$ combined with the kernel $K(\boldsymbol{x}, \boldsymbol{x}') = |\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$ defines a QSVM kernel. The intuition is that the quantum device is implicitly computing a rich similarity measure via its high-dimensional state space.
This is analogous to the classical kernel trick but potentially more powerful: **quantum kernels can potentially offer advantages over classical kernel methods, such as speedup, accuracy, flexibility, and robustness**.

Examples of Feature Map Circuits

There are many choices for $U(\boldsymbol{x})$. Common simple maps include:

Angle (or rotation) embedding:

For an n-dimensional feature vector $\boldsymbol{x} = (x_1, \ldots, x_n)$, apply single-qubit rotations $\mathrm{R}_X(x_i)$ or $\mathrm{R}_Y(x_i)$ to the ith qubit. For example:

$$U(\boldsymbol{x}) = \bigotimes i = 1^n R_Y(x_i) = R_Y(x_1) \otimes \cdots \otimes R_Y(x_n).$$

This maps x_i to an angle on the Bloch sphere. It produces product states (no entanglement).

More features

Amplitude encoding:

Encode the normalized data vector $\boldsymbol{x}$ directly into the amplitudes of the quantum state. For example, for 2 qubits one can map (x_1, x_2, x_3, x_4) (with $\sum x_i^2 = 1$) to $x_1|00\rangle + x_2|01\rangle + x_3|10\rangle + x_4|11\rangle$. This uses exponentially many amplitudes and can be powerful, but requires complex circuits.

Entangling feature maps:

Combine rotations with entangling gates. For instance, the ZZ feature map used in [Havlíček et al.] alternates single-qubit $R_Z(x_i)$ rotations with controlled-Z (CZ) gates between qubits. This creates entangled states whose overlaps can capture correlations in the data. In PennyLane, one could implement something like:

```
for i in range(n):
    qml.RZ(x[i], wires=i)
for i in range(n-1):
    qml.CNOT(wires=[i, i+1])
for i in range(n):
    qml.RZ(x[i], wires=i)
```

Polynomial feature map

These are inspired by classical polynomial kernels, one can encode quadratic or higher terms. For example, apply $R_Z(x_i), R_X(x_i)$ on each qubit and use multi-qubit rotations to create cross-terms . The choice of feature map affects the kernel. A richer (more entanglement) map may separate classes better but might also be harder to learn or require more quantum resources. In practice, one often starts with simple maps (e.g. AngleEmbedding) and experiments.

Kernel Estimation on Quantum Devices

Once $|\phi(\boldsymbol{x})\rangle$ is defined, we must compute $k(\boldsymbol{x}, \boldsymbol{x}')$ on actual quantum hardware. Typically this involves running a quantum circuit multiple times (shots) to estimate a probability. For the overlap kernel, one measures the probability of a certain outcome after applying $U(\boldsymbol{x})$ and $U(\boldsymbol{x}')^\dagger$. Alternative schemes include the SWAP test, which uses an ancilla qubit to directly estimate overlaps, but this requires extra gates. The above adjoint-circuit method only needs the same number of qubits as the feature map.

Short summary

In summary, the quantum kernel is given by inner products in a quantum-defined feature space. We emphasize: all computations on the quantum device yield kernel values (scalars). The heavy lifting of classification (optimizing α_i) remains a classical task, typically done by classical SVM software using the quantum-computed kernel matrix. This **hybrid** approach leverages quantum hardware where it's needed (kernel evaluation) and classical hardware for optimization.

Quantum Support Vector Machine Theory

We now combine the classical SVM formulation with quantum kernels. In essence, we run a classical SVM algorithm (e.g. solving the dual) but supply it with a quantum-computed kernel matrix. The resulting model is the same form:
$f(\boldsymbol{x}) = \text{sign}(\sum_i \alpha_i y_i K(\boldsymbol{x}_i, \boldsymbol{x}) + b)$, but K comes from a quantum device. The hope is that the quantum kernel captures data structure that a classical kernel might not.

QSVM Formulation via Quantum Kernels

Given training data $(\boldsymbol{x}_i, y_i)$, we choose a quantum feature map $U(\boldsymbol{x})$ and define the kernel $K_{ij} = |\langle \phi(\boldsymbol{x}i)|\phi(\boldsymbol{x}j)\rangle|^2$. Then we solve the binary type of SVM problems discussed last week, with $x_i^T x_j$ replaced by K_{ij}. That is:

$$\max \lambda \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \, K(\boldsymbol{x}_i, \boldsymbol{x}_j),$$

subject to $\sum_i \lambda_i y_i = 0$, $0 \leq \lambda_i \leq C$. After solving for λ_i, the decision function is

$$f(\boldsymbol{x}) = \text{sign}\Big(\sum i \lambda_i y_i \, K(\boldsymbol{x}_i, \boldsymbol{x}) + b \Big).$$

All kernel values $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ are estimated by the quantum device. Thus the QSVM is essentially a classical SVM with a learned quantum kernel.

Important training point

A key point is that training is still classical: one uses a conventional convex solver (e.g. libSVM) on the kernel matrix. The quantum part is in computing K_{ij}. This is sometimes called a quantum kernel estimator. Alternatively, one can approximate gradients of a parameterized kernel for optimization, but here we focus on the fixed-kernel case.

Why might a quantum kernel be better than a classical one?

The hope is that the Hilbert-space embedding is very high-dimensional (exponential), so that more functions (decision boundaries) become linear in that space. Replacing dot products by quantum overlaps is a generalization of classical kernels. In fact, Schuld argues that many near-term **quantum neural networks** are really kernel machines: "a lot of near-term and fault-tolerant quantum models can be replaced by a general support vector machine whose kernel computes distances between data-encoding quantum states" . Thus, one may as well solve directly in kernel form.

Quantum SVM Algorithms (large-scale vs NISQ)

There are two broad approaches in the literature for QSVMs:

Quantum Algorithms (fault-tolerant):

Early work by Rebentrost, Mohseni, and Lloyd (2014) formulated an SVM in terms of quantum linear algebra. They showed that one can invert the kernel matrix (a positive semidefinite matrix) using quantum algorithms (HHL algorithm) in time polylogarithmic in N and d . Concretely, they assumed quantum RAM (QRAM) access to data and used a quantum subroutine to solve the dual SVM as a linear system, yielding the vector of α_i in superposition. Under ideal conditions (sparse data, good condition number), this yields exponential speed-up over classical SVM training. However, it requires fully fault-tolerant quantum hardware and QRAM, making it more a theoretical result than a near-term implementation.

And NISQ Quantum Kernels

More recent work focuses on near-term devices. Here the idea is to compute the kernel matrix entries with a quantum circuit on current hardware or simulators, and then use a classical SVM solver for the rest. This does not claim an exponential runtime speed-up (indeed, the kernel matrix still has N^2 entries). Instead, it aims for a possible quality or feature-based advantage: the quantum feature map might separate data better than any known classical kernel. This approach has been demonstrated on small datasets (e.g. Iris) and studied theoretically. For example, Havlicek et al. showed on a toy problem that a quantum kernel can correctly classify points that a simple classical kernel cannot. However, other studies have found that for random data classical kernels often suffice, so the advantage is subtle. Research is ongoing on which problems truly benefit from quantum kernels.

Quantum neural network

Another variation is the quantum variational classifier, sometimes called a quantum neural network. Instead of precomputing a fixed kernel, one trains a parameterized quantum circuit to output labels. Interestingly, Schuld (2021) shows that variational quantum models, when trained by minimizing a loss, are mathematically equivalent to kernel machines with a particular kernel determined by the circuit . In fact, one can often find a kernel SVM that matches or outperforms the variational model. In practice, one can combine these: use a trainable quantum embedding $U(\boldsymbol{x}; \theta)$ with tunable parameters θ, and optimize θ to maximize the SVM classification accuracy. This is called a quantum kernel learning approach.

Quantum vs Classical SVM Performance

For our purposes, we concentrate on the case where the quantum feature map is fixed (or manually chosen), and we use classical SVM optimization on the resulting kernel.
Quantum SVMs are subject to the same generalization considerations as classical SVMs: overfitting can occur if the kernel is too flexible, and the choice of regularization C matters. One advantage is that quantum kernels can in principle provide very complex decision boundaries. On the other hand, estimating the kernel matrix accurately requires many quantum measurements (shots). Each kernel entry K_{ij} is obtained by sampling; statistical noise in kernel values can degrade SVM performance. Error mitigation techniques may be needed. Also, evaluating a kernel classically might in some cases simulate the quantum one if the circuit is not too complex; then no advantage is gained.

Comparing computational complexity

1. Classical SVM: Training roughly $O(N^3)$ time, inference $O(N_s)$ time per point, where N_s is number of support vectors.
2. Quantum kernel SVM: Kernel estimation takes $O(n_q N^2)$ time (where n_q is quantum circuit time per inner product) to compute all K_{ij}, plus classical $O(N^3)$. No asymptotic speed-up unless one uses quantum linear algebra (which still needs QRAM).
3. Quantum linear SVM: Potential $O(\log N)$ for training under ideal QRAM assumptions, inference also $O(\log N)$, at expense of large constant overhead.

Actual implementations

In practice on current hardware, QSVM speed is dominated by circuit execution time. However, QSVM experiments are valuable to test expressivity: for some data, a simple quantum feature map yields perfect classification where classical kernels fail.
Finally, it is worth noting that both the variational quantum classifier and the kernel QSVM ultimately hinge on how data is encoded into quantum states. Schuld summarizes that **the way that data is encoded into quantum states is the main ingredient that can potentially set quantum models apart from classical machine learning models**. Thus, a lot of focus is on designing powerful quantum embeddings and kernels. The SVM framework provides a clear way to evaluate them.

Practical Implementation with PennyLane

We now turn to hands-on implementation of QSVM using PennyLane, a Python library for quantum machine learning. PennyLane interfaces with quantum simulators and hardware and provides convenient utilities for kernels and optimization. In this chapter we will outline how to construct a QSVM: define a quantum feature map, compute the kernel matrix, and train a classical SVM using scikit-learn.

Setting up PennyLane

First, install PennyLane (pip install pennylane) and import necessary modules:

```
import pennylane as qml
from pennylane import numpy as np
from sklearn import svm
```

PennyLane uses devices to run quantum circuits. For QSVM experiments we can use the default.qubit simulator:

```
dev = qml.device("default.qubit", wires=n_qubits, shots=None)
```

Here n_qubits is chosen based on our feature map (often equal to data dimension or its log, etc.) shots=None means analytic probabilities (no sampling noise). For realistic experiments, set shots=100 or so to simulate finite sampling.

Defining Quantum Feature Maps in PennyLane

We must define a quantum circuit (QNode) that encodes input vectors. For example, for a 2-dimensional input $\boldsymbol{x} = (x_0, x_1)$, we might use AngleEmbedding:

```
@qml.qnode(dev)
def feature_map(x):
    qml.templates.AngleEmbedding(x, wires=[0,1])
    return [qml.expval(qml.PauliZ(0)), qml.expval(qml.PauliZ(1))]
```

This feature map returns expectation values (though for kernel computation we will use a different approach). Alternatively, we can directly define a unitary:

```
@qml.qnode(dev)
def U(x):
    qml.RY(x[0], wires=0)
    qml.RY(x[1], wires=1)
    return qml.state()
```

This QNode returns the quantum state (though qml.state() may require a special device). For kernel computation, we actually want to overlap two circuits, as shown below.

Computing Quantum Kernel Matrices

To train an SVM, we need the kernel matrix $K_{ij} = K(\boldsymbol{x}_i, \boldsymbol{x}_j)$. PennyLane provides qml.kernels.kernel_matrix, which takes two datasets and a kernel function. We must supply a function **kernel(x1,x2)** that returns the overlap of states.

Using the adjoint trick, we define:

```
@qml.qnode(dev)
def kernel_circuit(x1, x2):
    # encode x1
    qml.templates.AngleEmbedding(x1, wires=[0,1])
    # apply inverse embedding of x2
    qml.adjoint(qml.templates.AngleEmbedding)(x2, wires=[0,1])
    # return probability of \vert 00>
    return qml.probs(wires=[0,1])[0]
```

This returns $|\langle\phi(x_1)|\phi(x_2)\rangle|^2$ (the prob. of measuring all zeros).

We then set:

```
kernel = lambda a, b: kernel_circuit(a, b)
```

and compute kernel matrices. For training:

```
X_train = np.array([...])  # shape (N_train, 2)
Y_train = np.array([...])  # labels +1/-1
K_train = qml.kernels.kernel_matrix(X_train, X_train, kernel)
```

Training SVM with Precomputed Quantum Kernels

With the kernel matrix in hand, we use scikit-learn's SVM with a precomputed kernel. In Python:

```
clf = svm.SVC(kernel='precomputed')
clf.fit(K_train, Y_train)
y_pred = clf.predict(K_test)
```

The SVM solver will treat K_train[i,j] as the feature inner products. After fitting, one can retrieve support vectors via **clf.support** if desired. The prediction y_pred will be an array of +1/-1 predictions on the test set.
It is also possible to integrate PennyLane's differentiable capabilities by defining a parameterized kernel and optimizing parameters via gradient descent, but here we keep a fixed feature map.

Example: Kernel SVM in PennyLane

Below is a compact example that ties it together for a toy 2D dataset:

```
import pennylane as qml
from pennylane import numpy as np
from sklearn import svm

# Example dataset
X_train = np.array([[0.1, 0.2], [1.1, -0.4], [0.0, 0.9], [1.0, 0.5]])
Y_train = np.array([+1, +1, -1, -1])
X_test = np.array([[0.2, 0.1], [0.9, 0.0]])

# Define device and feature map
dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev)
def kernel_circuit(x1, x2):
    qml.templates.AngleEmbedding(x1, wires=[0,1])
    qml.adjoint(qml.templates.AngleEmbedding)(x2, wires=[0,1])
    return qml.probs(wires=[0,1])[0]

kernel = lambda a, b: kernel_circuit(a, b)

# Compute kernel matrices
K_train = qml.kernels.kernel_matrix(X_train, X_train, kernel_function=
K_test = qml.kernels.kernel_matrix(X_test, X_train, kernel_function=ke

# Train SVM
```

Discussion of Implementation

In practice, one must consider:

1. Noise and shots: If using real hardware or finite shots, each entry K_{ij} is a noisy estimate. It is common to run many shots (e.g. 1000) to reduce variance. One may also average results or use kernel averaging.
2. Data preprocessing: Classical kernels often require normalized or scaled inputs. Similarly, for quantum kernels, it can help to scale data so that features match the expected range of angles.
3. Computational cost: Computing an $N \times N$ kernel matrix on a quantum device takes $O(N^2)$ circuits. For large N this can be slow. One might use a subset of training data or low-rank approximations.
4. Hyperparameters: The QSVM has hyperparameters (e.g. SVM regularization C) that should be tuned by cross-validation. The choice of feature map is also a "hyperparameter" of sorts.

PennyLane implementations

In summary, implementing QSVM with PennyLane involves four steps:

1. choose a feature map and implement it as a QNode;
2. define a kernel function via that QNode;
3. compute kernel matrices with qml.kernels;
4. train/predict using an SVM with the precomputed kernels.

Steps in Quantum Kernel SVM

1. Data Encoding (Feature Map Generation)
   - The classical data (x) is encoded into a quantum state using a quantum feature map (quantum circuit).
2. Quantum Kernel Computation
   - The kernel matrix is computed using a quantum computer by measuring the inner product of quantum states.
3. Classical SVM Training
   - The quantum kernel matrix is used in a classical SVM algorithm to find the optimal decision boundary.
4. Prediction
   - New data is classified using the trained SVM model with the quantum kernel.

Iris Dataset

```
import pennylane as qml
from pennylane import numpy as np
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
num_qubits=4
device=qml.device('default.qubit',wires=num_qubits)


@qml.qnode(device)
def qnode(inputs,weights):
    qml.templates.AngleEmbedding(inputs,
                                 wires=range(num_qubits),
                                 rotation='Y') # encode the inputs int
    qml.adjoint(qml.AngleEmbedding(features=weights,
                                   wires=range(num_qubits),
                                   rotation='Y')) # applies the inver
    return qml.probs(wires=range(num_qubits))


def qkernel(inputs,weights):
    return np.array([[qnode(input,weight)[0] for weight in weights] fo


# Fit the model
```

Qiskit implementation

A similar implementation on Qiskit is given here (you need to have installed Qiskit as well).

```
from qiskit import BasicAer
from qiskit.utils import QuantumInstance
from qiskit.circuit.library import ZZFeatureMap
from qiskit_machine_learning.algorithms import QSVC
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load and preprocess dataset
dataset = load_iris()
X = dataset.data
y = dataset.target

# For simplicity, we'll only classify between two classes
X = X[y != 2]
y = y[y != 2]

# Standardize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.

# Define quantum feature map
```

Plans for next week

1. Summary of quantum support vector machines
2. Quantum neural networks